Tack

README summary. A lean, fast, local desktop harness for coding agents. It runs Claude Code and Codex on your existing plans.

Principles

- Local only: no cloud, accounts or telemetry.
- No feature ships unless people use it to code with agents.
- Lean context: start from default Claude Code and Codex prompts, strip what hurts quality or speed.
- Speed comes first.

Stack

Part	What	Where
Shell	Tauri 2, thin Rust layer: window, dialogs, sidecar launch	`src-tauri/`
Sidecar	Bun, WebSocket server, SQLite + JSONL transcripts, agent sessions	`server/`
UI	React 19 with compiler, Vite, Tailwind v4, Streamdown	`web/`
Protocol	Typed requests and events shared by UI and sidecar	`shared/protocol.ts`

Claude runs through the official `claude` binary (Agent SDK) and uses your Claude login. macOS only for now.

Develop

Requires Bun, Rust and a logged-in `claude` CLI.

- `bun install`, then `bun run dev` for the desktop app with UI hot reload.
- Browser UI: `TACK_DEV=1 TACK_PORT=7357 bun run dev:server` and `bun run dev:web` (http://localhost:1420).
- `bun run build` bundles to `src-tauri/target/release/bundle/`.

How it works

- One warm `claude` process per active thread; idle ones close after 15 minutes and resume the session on the next message.

- Streaming text is coalesced in memory and sent to the UI at most every 33 ms; only finished items are written, one JSONL line each.
- Virtualized transcript; only the streaming row re-renders.
- Queue and steer: `↵` queues a message, `⌘↵` interrupts and sends now, `Esc` stops the agent.
- Permissions: full access by default; "Ask first" prompts for each tool call not already allowed in Claude settings.